

112550200 鄭武謙

Problem 1

It calculates the difference (often denote as δ) between the reward receive plus the discounted estimated value of the next state and the current state's estimated value. This δ is basic build block for Generalized Advantage Estimation. By computing it at each time step, the code gathers the error signals that, when discounted and summed later, form the overall advantage estimate.

In short, Lines 25-29 set up the computation that will later be used to recursively accumulate advantages. This step is essential because it bridges the gap between raw rewards and value function predictions, allowing the algorithm to combine both Monte Carlo and bootstrapped information for a more stable, low-variance advantage estimate.

Problem 2

The function `train_critic_td` trains the critic network using TD learning. It uses a one-step TD target and minimizes the mean squared error between the current value estimates and the TD targets (here's how each line translates into mathematical terms):

$$\text{criterion} = \text{nn.MSELoss}() \Rightarrow L = \frac{1}{N} \sum_{i=1}^N (\hat{V}(s_i) - y_i)^2$$

$$\text{values} = \text{value_net}(\text{states}).\text{squeeze}() \Rightarrow \hat{V}(s) \text{ for } s \in \{s_1, s_2, \dots, s_N\}$$

$$\text{next_values} = \text{value_net}(\text{next_states}).\text{squeeze}().\text{detach}() \Rightarrow \hat{V}(s') \text{ for } s' \in \{s'_1, s'_2, \dots, s'_N\}$$

$$\text{target_value} = \text{rewards} + \gamma * \text{next_value} * (1 - \text{done}) \Rightarrow y = r + \gamma \hat{V}(s') (1 - d)$$

$$\text{loss} = \text{criterion}(\text{values}, \text{target_values}) \Rightarrow L = \frac{1}{N} \sum_{i=1}^N (\hat{V}(s_i) - y_i)^2$$

$$\text{loss.backward}() \Rightarrow \nabla_{\theta} L$$

$$\text{optimizer.step}() \Rightarrow \theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

Problem 3

We can normalize the advantage estimates before using them in policy updates. In many actor-critic algorithms, the raw advantage values can vary widely in scale, which may cause unstable learning. By normalizing, we can ensure that the updates are more consistent and stable.

Take an example. After computing the advantages, we can add a normalization step such as

$$\text{advantage} \leftarrow \frac{\text{advantage} - \mu}{\sigma + \epsilon}$$

By doing so, we can reduce variance in the gradient updates and improve the convergence of the learning algorithm.